DEMO 01 · INDUSTRIAL CONTROL

Autonomous Control That Fails Safe

A reactor-style cascade controller that issues real cooling commands — and trips to a safe state in a single tick when an alarm fires.

AUTONOMOUS-MOCK *Autonomous logic, local mock only*

AUTHOR Rakovskyi Dmytro · VERSION Full-run demo 01 · DATE 2026 · REPOSITORY github.com/rakovpublic/jneopallium

This is the only demo in the suite trusted with autonomous authority, and it earns that trust the hard way: by behaving conservatively under stress. Fed an OPC-UA-style stream of temperature, flow, and valve feedback, it runs a cascade control loop toward a cooling target — but the moment a high-temperature alarm appears, a dedicated safety layer overrides everything and forces a fail-safe command within one tick. The autonomy is real; the brakes are structural.

What it is

Demo 01 models continuous process control for a reactor-like asset, `reactor-a`. Its safety ceiling is **AUTONOMOUS-MOCK**: it issues genuine control intent — open, close, hold, trip — but against a simulated actuator, so the full closed loop can be exercised end-to-end without touching hardware. The network is deliberately small: five layers, sized 4·3·2·2·3, which is the least depth that produces a safe cascade decision.

The point is not throughput. It is to show that a neural controller can hold autonomous authority over a process and still be provably bounded, because the boundary lives inside the decision path rather than wrapped around the outside.

How it is wired

Signals flow up from the sensor edge to a decision and back down to a command, with a safety layer standing between the control decision and anything that leaves the process.

Figure *Five layers. The safety gate sits between the control decision and the valve command, and it can override both.*

The signal vocabulary is concrete and typed: `TemperatureSignal` and `FlowSignal` carry process values, `AlarmSignal` carries the high-temperature condition, `ControlDemandSignal` and `ValveCommandSignal` carry the controller's intent, and `SafetyVetoSignal` carries the safety layer's override. A layer expecting a flow reading cannot be handed an alarm by accident.

What it does

Three behaviours define the demo, and together they make the autonomy defensible.

- **Normal operation** — with process values in range, the controller moves the valve command toward its cooling target through a standard cascade rule.
- **Alarm** — when a high-temperature alarm fires, the safety layer forces a fail-safe command, overriding whatever the control layer wanted.
- **Manual override** — when a human holds the command, the result aggregator records the held/rejected command rather than silently dropping it.

> **SAFETY**
> The defining property: a forced alarm produces a fail-safe command within a single tick. The safety response is not averaged in over time — it is immediate and it is structural.

The evidence

The demo runs deterministically for 200 ticks and writes a full result row per tick plus an audit decision stream.

VERIFICATION EVIDENCE	
Ticks	200 / 200, all rows present
Forced alarm @ t5	fail-safe command within **1 tick**
Manual override @ t10	command **held** and recorded
Verdict	**PASS**

How to run it

The demo runs through the real Jneopallium local path — the framework entry point loads the model JAR and a context, not a bespoke harness.

1 Build

Build the worker and the demo model JAR.

```
mvn -q -pl worker -am package
```

2 Run via the framework entry point

Point Entry at the model JAR (both on the classpath and as the file: URL) and the demo context.

```
java -cp demo-model.jar \
  com.rakovpublic.jneuropallium.worker.application.Entry \
  local file:///path/to/demo-model.jar \
  com.rakovpublic.jneuropallium.worker.demo.fullrun.runtime.DemoJsonContext \
  context.json
```

3 Inspect

Read the per-tick results and the audit stream.

```
ls target/jneopallium-fullrun-demos/demo-01-industrial-control/
```

Why AUTONOMOUS-MOCK

Autonomous authority is granted here only because the actuator is simulated and the fail-safe is non-negotiable. To take this controller live, you replace the mock sensor edge with a real OPC UA client while preserving two things exactly — the typed signal contract and the safety ceiling. The cascade rule you verified against a deterministic mock behaves identically against a live process value, and the one-tick fail-safe still fires. You verify the safety once; swapping the inputs does not ask you to verify it again.